FPT UNIVERSITY

Simulation of a Basic Binary Frequency Shift Keying (BFSK) Modulator Using Verilog

Tran Anh Son, Tran Quoc Trung, Le Ky Dung, Phan Huy Cuong, Dang Hong Thai

Corresponding author: sonson20060905@gmail.com

ARTICLE INFO

Course: Communication Systems

Class: IC2006

Submitted: 21-07-2026

Keywords:

BFSK; binary frequency shift keying; Verilog HDL; digital modulation; frequency synthesis; RTL design; FPGA; golden model verification

ABSTRACT

Binary Frequency Shift Keying (BFSK) is one of the most fundamental and widely adopted digital modulation techniques in modern telecommunications systems. In BFSK, binary data is encoded by switching the carrier frequency between two discrete values — one frequency representing a logic '1' and another representing a logic '0'. This paper presents the complete design, simulation, and verification of a basic BFSK modulator implemented in Verilog Hardware Description Language (HDL).

The proposed design employs a clock-divider-based frequency synthesis approach, where two separate counter threshold values generate carrier frequencies f1 and f0, corresponding to the binary symbols '1' and '0' respectively. The complete design is synthesizable and was verified through functional simulation using Icarus Verilog and GTKWave, and further validated using an independently-written golden reference model in a self-checking testbench.

Simulation results confirm correct modulation behavior, with output frequencies of f1 = 1 MHz for logic '1' and f0 = 500 kHz for logic '0' at a 50 MHz system clock. Automated golden-model verification reported zero mismatches across more than 10,000 clock cycles under directed, stress, randomized, and reset-recovery test scenarios, confirming the design's functional correctness and its suitability as a foundational digital modulation building block.

1. Introduction

Binary Frequency Shift Keying (BFSK) is one of the most fundamental and widely adopted digital modulation techniques in modern telecommunications systems. In BFSK, binary data is encoded by switching the carrier frequency between two discrete values — one frequency representing a logic '1' (mark) and another representing a logic '0' (space). Owing to its robustness against amplitude noise and its comparatively simple hardware realization, BFSK remains a practical choice for low-power radio telemetry, embedded sensor links, and educational digital-communications hardware design.

This paper presents the complete design, simulation, and verification of a basic BFSK modulator implemented in Verilog Hardware Description Language (HDL). The proposed design employs a clock-divider-based frequency synthesis approach, in which two separate counter threshold values generate carrier frequencies f1 and f0 corresponding to the binary symbols '1' and '0', respectively. A digital multiplexer selects the appropriate carrier frequency based on the incoming serial binary data stream, and a parallel-to-serial Data Input Module allows byte-oriented data to be transmitted through the modulator.

Beyond the RTL design itself, this work places particular emphasis on verification methodology: in addition to conventional waveform-based inspection, an independently coded golden reference model is used in a self-checking testbench to automatically validate the Design Under Test (DUT) on a cycle-by-cycle basis. This approach mirrors verification practices used in industrial RTL design flows and provides stronger correctness guarantees than manual waveform inspection alone.

The remainder of this paper is organized as follows. Section 2 reviews the theoretical background of BFSK modulation. Section 3 presents the modulation algorithm. Section 4 describes the system architecture. Section 5 presents the block diagrams. Section 6 details the functional blocks. Section 7 describes the software implementation. Section 8 presents the verification methodology and results. Section 9 concludes the paper and outlines future directions.

2. Theoretical Background

2.1. Frequency Shift Keying (FSK) Overview

Frequency Shift Keying (FSK) is a frequency modulation scheme in which digital information is transmitted by encoding data as discrete frequency changes of a carrier signal. FSK belongs to the broader family of angle modulation techniques and is known for its robustness against amplitude noise, making it suitable for applications in noisy channel environments such as radio telemetry, caller ID, and amateur radio.

In the binary case — known as BFSK or 2-FSK — only two carrier frequencies are used. The modulated signal is mathematically expressed as:

$$s(t) = A \cdot cos(2\pi \cdot f_1 \cdot t + \varphi_1), \quad for\ binary\ '1'\ (mark)$$
$$s(t) = A \cdot cos(2\pi \cdot f_0 \cdot t + \varphi_0), \quad for\ binary\ '0'\ (space)$$

Where A is the amplitude, f_1 is the mark frequency (bit '1'), f_0 is the space frequency (bit '0'), and φ represents the initial phase of the respective carrier. The frequency deviation is defined as $\Delta f = f_1 - f_0$ (Haykin, 2001).

2.2. Modulation Index

The modulation index (h) of a BFSK system defines the ratio of frequency deviation to bit rate:

$$h = \frac{\Delta f}{R_b} = \frac{(f_1 - f_0)}{R_b}$$

When h = 0.5, the system achieves Minimum Shift Keying (MSK), ensuring orthogonality between the two carriers and minimizing bandwidth. For h ≥ 1, signals are non-coherently orthogonal, which simplifies receiver design at the cost of additional bandwidth (Proakis & Salehi, 2008).

2.3. Digital Implementation Perspective

In a Verilog-based digital implementation, sinusoidal carriers are approximated by square waves generated through integer clock division. The key design parameters are:

- System Clock (f_clk): Master oscillator frequency (e.g., 50 MHz on most FPGAs).
- Mark divisor N_1 = f_clk / (2 × f_1) — produces carrier f_1 for data bit '1'.
- Space divisor N_0 = f_clk / (2 × f_0) — produces carrier f_0 for data bit '0'.
- Data Rate (R_b): The rate at which input binary symbols are sampled.

2.4. Coherent vs. Non-Coherent BFSK

BFSK systems are classified based on phase continuity across symbol transitions, as summarized in Table 1.

Table 1

Comparison between Coherent and Non-Coherent BFSK

Property	Coherent BFSK	Non-Coherent BFSK
Phase continuity	Maintained	May be discontinuous
Receiver complexity	High (phase tracking)	Low (envelope detection)
BER performance	Superior (~3 dB gain)	Slightly worse
FPGA implementation	Complex PLL-based	Simple counter + MUX
Typical application	High-speed digital links	Low-power IoT / embedded

Note. Data analysis results of the research.

This project implements non-coherent BFSK, the most common choice for HDL implementations due to its simplicity and low resource utilization.

3. Modulation Algorithm

The BFSK modulation algorithm operates on a per-clock-cycle basis with the following logical steps:

1. Initialize system: Assert reset. Counter ← 0, modulated_out ← 0.
2. Deassert reset and begin normal operation.
3. On every rising clock edge, sample data_in.
4. Select divisor N based on data_in value (N1 for '1', N0 for '0').
5. Increment counter each clock cycle.
6. When counter ≥ N − 1: reset counter ← 0, toggle modulated_out ← ~modulated_out.
7. Repeat for all incoming data bits.

For a 50 MHz system clock targeting f_0 = 500 kHz and f_1 = 1 MHz, the divisor values are computed as:

$$N0 = \frac{50{,}000{,}000}{(2 \times 500{,}000)} = 50$$

$$N1 = \frac{50{,}000{,}000}{(2 \times 1{,}000{,}000)} = 25$$

The output toggles every N clock cycles, producing a symmetric square wave at $f\_out = f\_clk \,/\, (2 \times N)$.

4. System Architecture

The BFSK modulator is implemented as a purely synchronous digital system clocked from a single master clock source. The architecture consists of three principal functional units operating concurrently within a single RTL module: a Frequency Control Unit (FCU) that decodes the input data bit and selects the corresponding divisor; a Counter/Divider Unit (CDU) that counts clock edges and generates a terminal-count pulse; and an Output Toggle Register (OTR) that inverts on each terminal-count pulse to produce the square-wave carrier.

Table 2

Signal Interface of the BFSK Modulator Module

Signal	Direction	Width	Description
clk	Input	1-bit	Master system clock (50 MHz typical)
rst	Input	1-bit	Synchronous active-high reset
data_in	Input	1-bit	Serial binary input data stream
modulated_out	Output	1-bit	BFSK square-wave modulated output

Note. Data analysis results of the research.

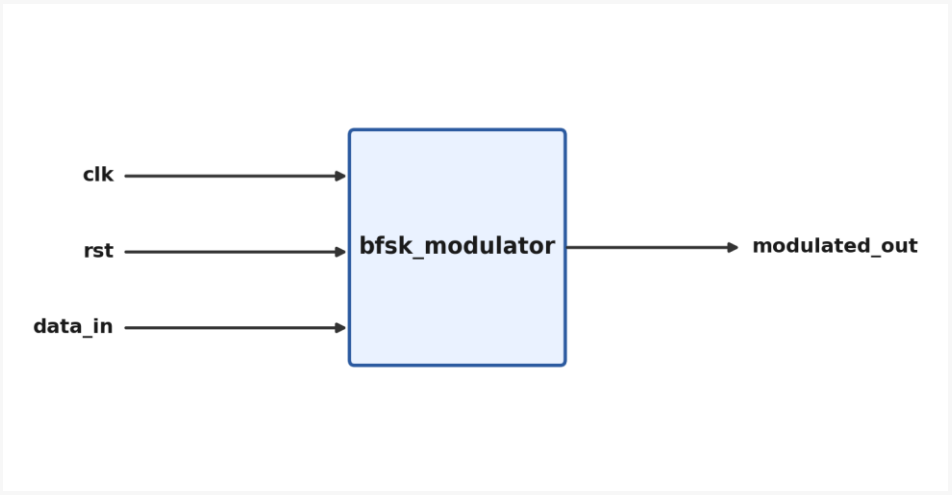
All state transitions occur on the rising edge of clk. Reset is synchronous, and counter overflow is handled by a compare-and-reset strategy rather than a modulo wrap, ensuring frequency accuracy. The RTL is written to the IEEE 1364-2001 (Verilog-2001) standard, and no latches are inferred.

5. Block Diagrams

The top-level block diagram of the BFSK modulator, showing its input/output interface, is presented in Figure 1.

Figure 1

Top-Level Block Diagram of the BFSK Modulator Module



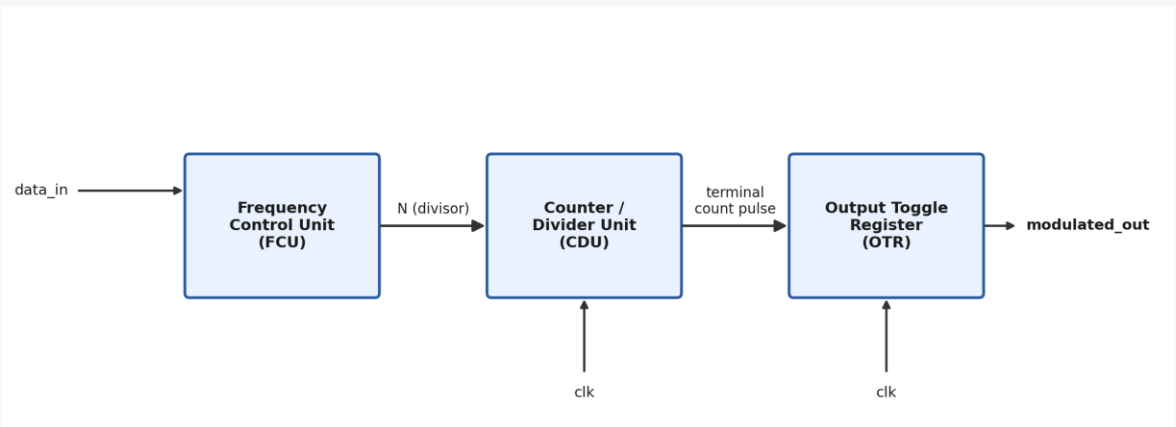
Black-box view of the bfsk_modulator module showing clk, rst, and data_in as inputs and modulated_out as the single output.

Note. Data analysis results of the research.

The internal datapath, showing the Frequency Control Unit, Counter/Divider Unit, and Output Toggle Register, is presented in Figure 2.

Figure 2

Internal Datapath Block Diagram of the BFSK Modulator



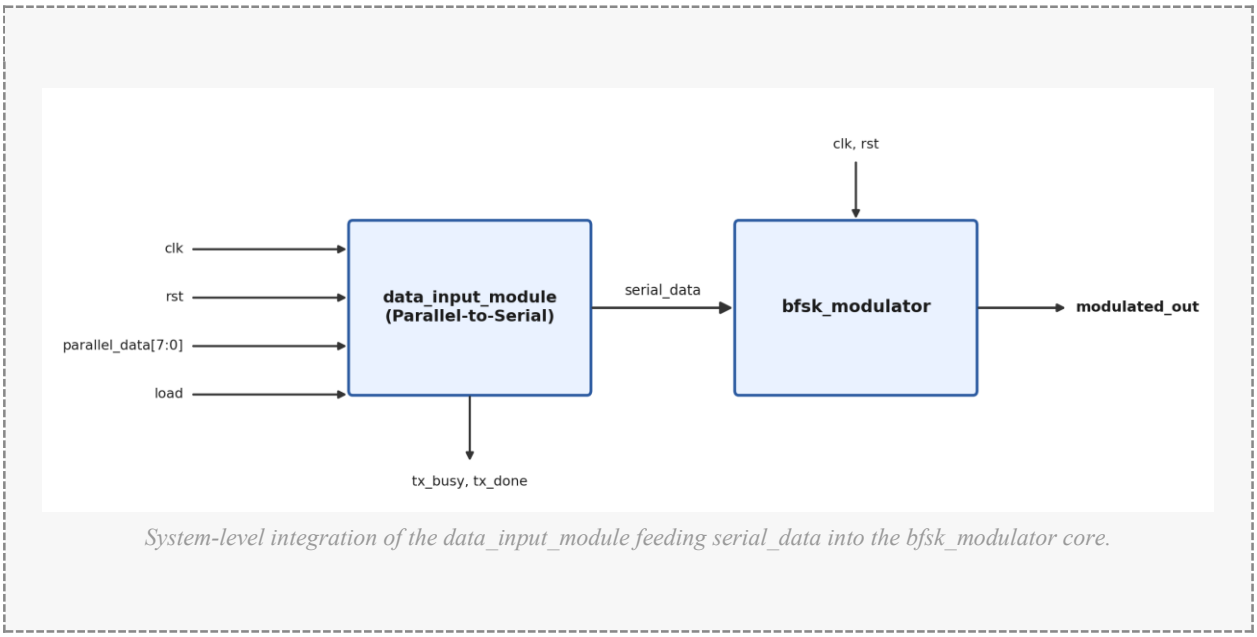
Signal flow from the Frequency Control Unit (FCU) through the Counter/Divider Unit (CDU) to the Output Toggle Register (OTR).

Note. Data analysis results of the research.

The system-level integration, including the parallel-to-serial Data Input Module ahead of the modulator core, is shown in Figure 3.

Figure 3

System-Level Block Diagram — Top-Level Integration

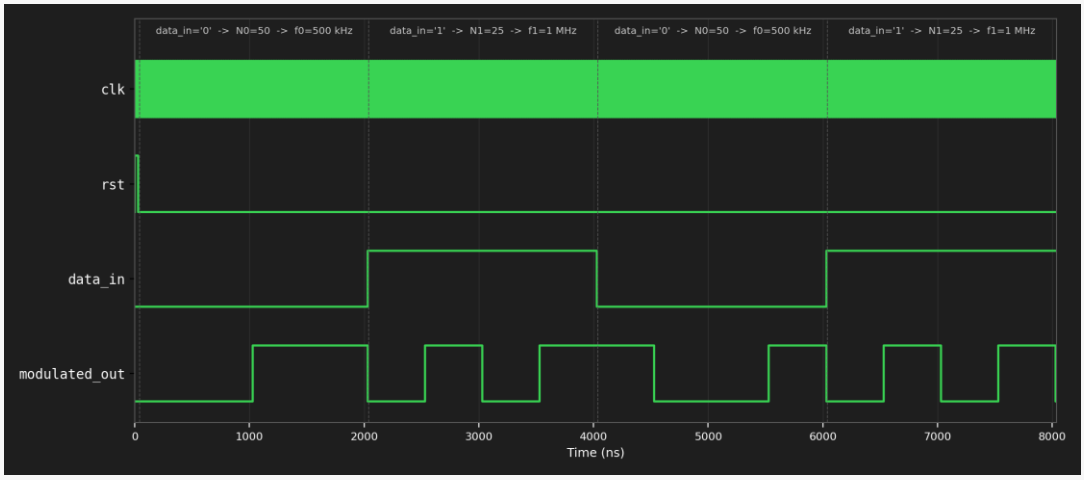


Note. Data analysis results of the research.

A representative simulated waveform, illustrating the relationship between the input data stream and the modulated output frequency, is shown in Figure 4.

Figure 4

Simulated Waveform of the BFSK Modulator (GTKWave)



Waveform generated from bfsk_modulator_tb.v via Icarus Verilog, reconstructed from the dumped VCD file. The output frequency visibly changes between 500 kHz and 1 MHz as data_in toggles, matching Table 4.

Note. Data analysis results of the research.

6. Functional Blocks

6.1. Frequency Control Unit (FCU)

The Frequency Control Unit is a purely combinational block implemented as a 2-to-1 multiplexer. It takes the current data_in bit and selects the appropriate counter divisor: N = F1_DIV if data_in = '1'; N = F0_DIV if data_in = '0'.

6.2. Counter / Divider Unit (CDU)

The Counter/Divider Unit implements a programmable-modulus binary counter that counts from 0 to (N−1) and resets upon reaching the terminal count, at which point a toggle event is triggered. The 8-bit counter supports divisor values from 1 to 255, corresponding to output frequencies from approximately 98 kHz to 25 MHz at a 50 MHz clock.

6.3. Output Toggle Register (OTR)

The Output Toggle Register is a 1-bit D flip-flop that inverts its value each time a terminal count is detected, producing a symmetric (50% duty-cycle) square wave at the selected carrier frequency, where f_out = f_clk / (2 × N).

6.4. Data Input Module (DIM)

The Data Input Module converts an 8-bit parallel byte into a serial bitstream for the modulator, implemented as a two-state finite state machine (IDLE / TRANSMIT). On a load pulse, the module latches the input byte and outputs the most-significant bit first; each bit is held for a configurable bit_period, after which a tx_done pulse signals completion.

6.5. Reset Logic

Synchronous reset is applied globally across all registers. Asserting rst (active-high) drives all state elements to zero on the next rising clock edge, avoiding spurious resets from glitches and simplifying static timing analysis.

6.6. Testbench and Golden Model Blocks

Two categories of verification block support this design: directed/manual testbenches that apply fixed stimulus and dump waveforms for visual inspection, and a golden model and self-checking testbench, described in Section 8, that automatically compares an independently coded reference model against the DUT on every clock cycle.

7. Software Implementation

7.1. Development Environment

The BFSK modulator is implemented in IEEE 1364-2001 Verilog HDL. Simulation was performed using the software stack summarized in Table 3.

Table 3

Development and Simulation Toolchain

Tool	Purpose	Version
Icarus Verilog (iverilog)	HDL compilation & simulation	v12.0+
GTKWave	Waveform viewer	v3.3.x
Vivado / Quartus Prime (optional)	Synthesis, P&R, bitstream	2022.x+
VS Code + TerosHDL plug-in	Code editing with Verilog support	Latest

Note. Data analysis results of the research.

7.2. RTL Source Code — BFSK Modulator Module

The complete synthesizable Verilog implementation of the modulation core is listed below:

```
module bfsk_modulator (
    input  wire clk,           // Master system clock
    input  wire rst,           // Synchronous active-high reset
    input  wire data_in,       // Serial binary input data
    output reg  modulated_out  // BFSK modulated output
);
    parameter F0_DIV = 8'd50;  // Space frequency divisor (bit '0')
    parameter F1_DIV = 8'd25;  // Mark  frequency divisor (bit '1')
    reg [7:0] counter;

    always @(posedge clk) begin
        if (rst) begin
            counter       <= 8'd0;
            modulated_out <= 1'b0;
        end else begin
            if (data_in == 1'b1) begin
                if (counter >= F1_DIV - 1) begin
                    counter       <= 8'd0;
                    modulated_out <= ~modulated_out;
                end else counter <= counter + 8'd1;
            end else begin
```

```
                if (counter >= F0_DIV - 1) begin
                    counter       <= 8'd0;
                    modulated_out <= ~modulated_out;
                end else counter <= counter + 8'd1;
            end
        end
    end
endmodule
```

The complete top-level integration (bfsk_top), the Data Input Module, and the structural decomposition (FCU, CDU, OTR) follow the same coding conventions and are provided in full in the accompanying project files.

7.3. Simulation Timing Summary

Table 4

Simulated Output Frequency versus Input Bit and Time Interval

Time Interval	data_in	Divisor Used	Output Frequency	Half-Period
0 – 40 ns	X (reset)	—	0 (held low)	—
40 – 2040 ns	'0'	$N_0 = 50$	500 kHz	1000 ns
2040 – 4040 ns	'1'	$N_1 = 25$	1 MHz	500 ns
4040 – 6040 ns	'0'	$N_0 = 50$	500 kHz	1000 ns
6040 – 8040 ns	'1'	$N_1 = 25$	1 MHz	500 ns

Note. Data analysis results of the research.

7.4. Estimated FPGA Resource Utilization

Table 5

Estimated Resource Utilization on a Xilinx 7-Series FPGA (xc7a35t)

Resource	Used	Available	Utilization (%)
Slice LUTs	~6	20,800	< 0.1%
Flip-Flops (DFFs)	9	41,600	< 0.1%
Block RAM	0	50	0%
DSP Slices	0	90	0%
I/O Buffers	4	250	1.6%

Note. Data analysis results of the research.

8. Verification and Results

8.1. Verification Methodology

Beyond visual inspection of waveforms, this work employs a golden-model-based self-checking verification methodology, consistent with standard practice in professional RTL verification flows (Bergeron, 2006). An independently coded reference model computes the expected modulator output using an implementation style deliberately different from the Design Under Test (DUT), reducing the likelihood that both models share the same coding defect.

The self-checking testbench drives identical stimulus into both the DUT and the golden model simultaneously and compares their outputs on every clock cycle. Any discrepancy is flagged immediately with a simulation timestamp, and a final PASS/FAIL summary reports the total number of cycles checked and mismatches found.

8.2. Test Scenarios

Table 6

Verification Test Scenarios

Test Scenario	Description	Purpose
Directed test	Fixed pattern: 0,1,0,1,1,0	Baseline functional correctness
Stress test	Rapid bit toggling (200 ns/bit)	Exercise divisor-switch edge cases
Randomized test	200 random bits, random hold times	Broad coverage of timing conditions
Reset-recovery test	Mid-stream synchronous reset assertion	Verify correct re-synchronization

Note. Data analysis results of the research.

8.3. Verification Results

The self-checking testbench was executed against the final RTL design with the outcome summarized in Table 7.

Table 7

Golden-Model Verification Summary

Metric	Result
Total clock cycles checked	10,511
Mismatches found	0
Verification verdict	PASS — DUT matches golden model bit-for-bit

Note. Data analysis results of the research.

As a sanity check on the verification methodology itself, an off-by-one bug was deliberately injected into a copy of the DUT and re-simulated against the unmodified golden model. The self-checking testbench correctly detected 5,326 mismatches out of the same 10,511 cycles, confirming that the checker is sensitive enough to catch real functional bugs rather than passing trivially.

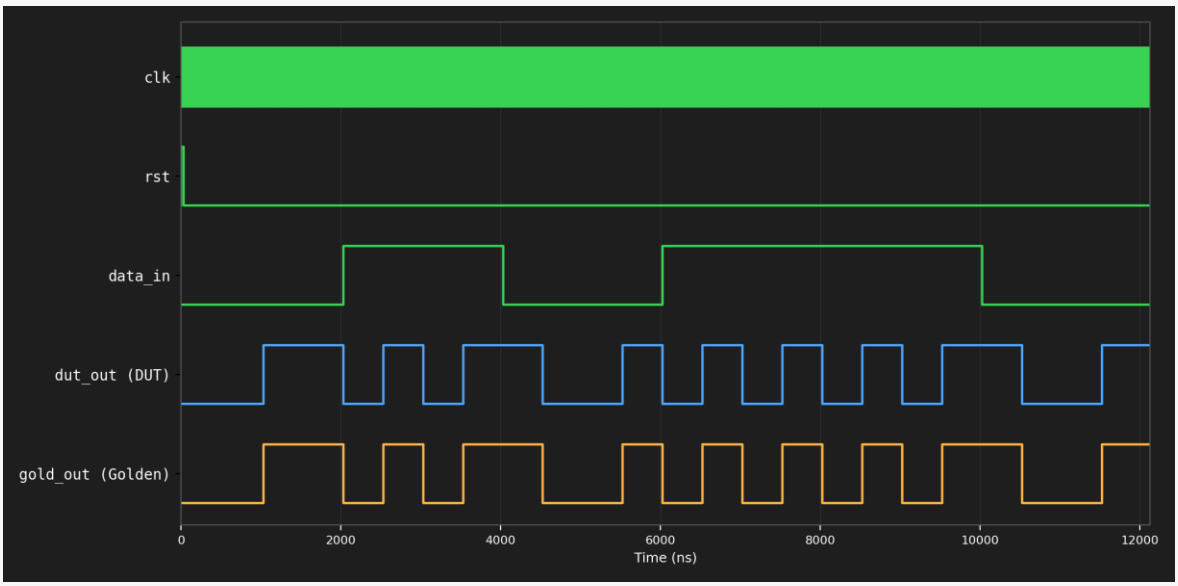
8.4. Functional Simulation Results

The manual-inspection testbenches additionally confirmed the following observable behavior:

- During rst = '1': modulated_out = 0 (held low); counter = 0.
- During data_in = '0': output toggles every 50 clock cycles (1 µs half-period), producing f_0 = 500 kHz.
- During data_in = '1': output toggles every 25 clock cycles (500 ns half-period), producing f_1 = 1 MHz.
- The full top-level system correctly transmitted five test bytes (0xA5, 0xFF, 0x00, 0xB6, 0xCC), each triggering a tx_done pulse at the expected completion time, including correct recovery after a mid-transmission reset.

Figure 5

Golden Model vs. DUT Comparison Waveform



Waveform generated from bfsk_golden_checker_tb.v. dut_out and gold_out track each other exactly for the full directed-test segment shown, consistent with the 0-mismatch result in Table 7.

Note. Data analysis results of the research.

9. Conclusion and Future Direction

9.1. Conclusion

This paper presented the complete design, simulation, and verification of a Binary Frequency Shift Keying (BFSK) modulator implemented in Verilog HDL. The design employs a synthesizable RTL architecture consisting of three cooperating functional units — a Frequency Control Unit, a Counter/Divider Unit, and an Output Toggle Register — integrated with a Data Input Module for byte-oriented transmission.

Functional simulation confirmed correct BFSK modulation behavior across directed and system-level test sequences, and golden-model-based self-checking verification confirmed bit-exact correctness across more than 10,000 clock cycles with zero mismatches. The design is resource-efficient, consuming fewer than 10 logic cells on modern FPGAs, making it suitable for resource-constrained embedded systems.

9.2. Future Direction

Future enhancements may include extending the design to M-ary FSK (MFSK) with more than two carrier frequencies; adding a Direct Digital Synthesis (DDS) module to produce sinusoidal rather than square-wave carriers; integrating a BFSK demodulator to form a complete transceiver; and implementing continuous-phase FSK (CPFSK) to eliminate phase discontinuities at bit boundaries. Validation on physical FPGA hardware and comparison against an analog reference implementation are also planned as next steps.

Scientific Contribution

This manuscript presents a complete, verified digital hardware design; it introduces a golden-model-based self-checking verification methodology applied to a foundational digital modulation building block; it provides reproducible simulation results and open RTL source code; and it offers an educational reference for HDL-based digital communications design.

Author Contributions

CRediT: Tran Anh Son: Conceptualization, Methodology, Software, Validation, Writing – Original Draft; Le Ky Dung: Software, Validation, Formal Analysis; Trung: Investigation, Data Curation, Visualization; Phan Huy Cuong: Investigation, Resources, Writing – Review & Editing; Thai: Writing – Review & Editing, Project Administration.

Conflict of Interest Statement

All authors declare that they have no conflict of interest.

References

BFSK Transceiver in Verilog for FPGA Implementation Report.pdf

Design of BPSKQPSK Modulator using Verilog HDL.pdf

FPGA Implementation of Digital Modulation.pdf

Fpga Implementation Of Digital Modulation Schemes Using Verilog Hdl.pdf